Mathematical Understanding of MLPs and the Backpropagation Algorithm

Moin Mostakim

Outline

1. Overview: MLP and Notation
2. Forward Pass: Matrix Form
3. Backpropagation: Scalar to Matrix
4. Putting it together
5. Derivations / Appendix

Multilayer Perceptron (MLP) – overview

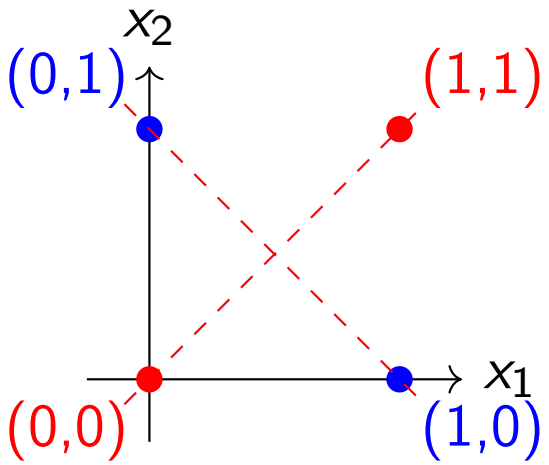
- Feedforward neural network with L layers (input layer 0, output layer L).
- Each hidden layer l has weights $W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$, bias $b^{[l]} \in \mathbb{R}^{n_l}$.
- Activations: $a^{[0]} = x$ (input). For $l \geq 1$:

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$$
$$a^{[l]} = \sigma^{[l]}\left(z^{[l]}\right)$$

- For a mini-batch of size m, stack $a^{[l]}$ as columns: $A^{[l]} \in \mathbb{R}^{n_l \times m}$.

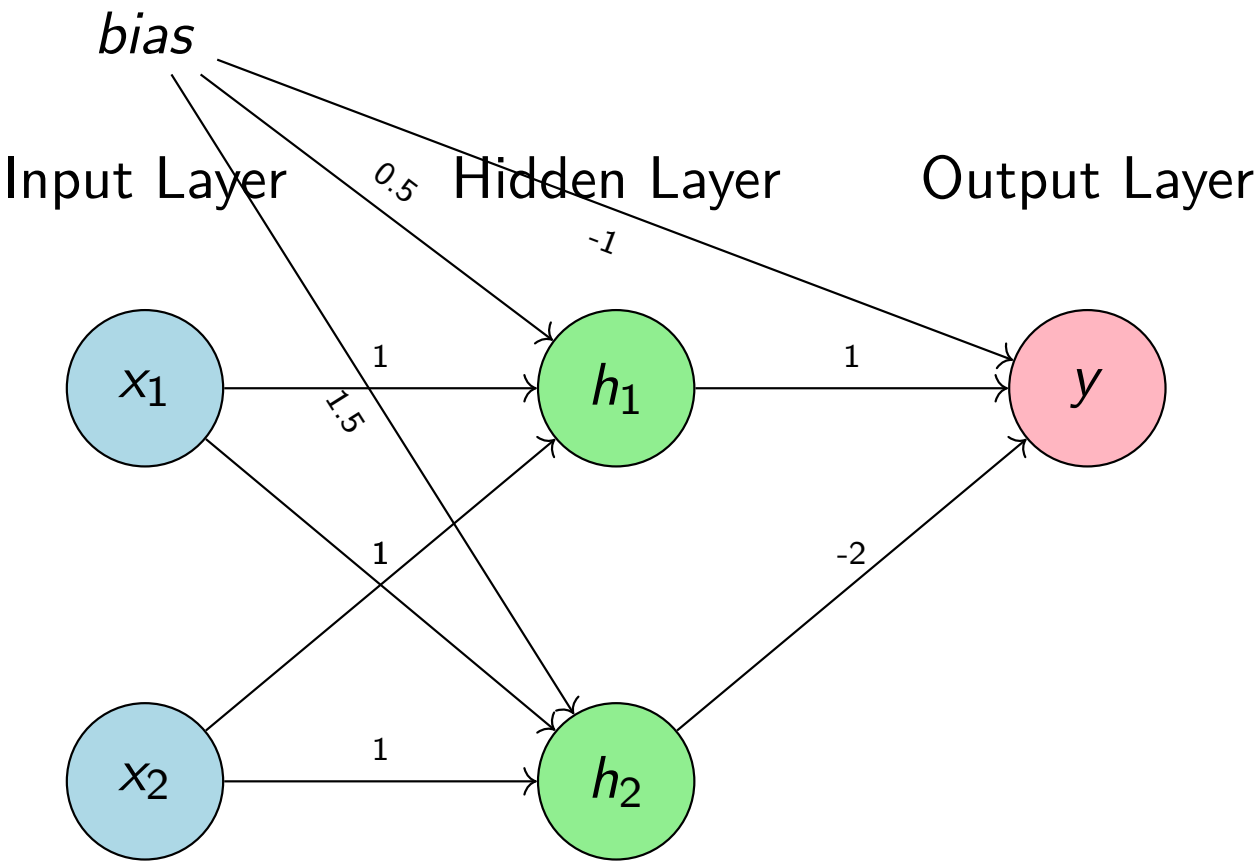
The XOR Problem

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



- Non-linearly separable problem
- Single-layer perceptron cannot solve XOR
- Requires hidden layer for solution

Neural Network Architecture for XOR



Weight Assignment and Activation Function

Weight Matrix Notation

- Input to Hidden weights: $W^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$
- Hidden biases: $b^{(1)} = \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix}$
- Hidden to Output weights: $W^{(2)} = \begin{bmatrix} 1 & -2 \end{bmatrix}$
- Output bias: $b^{(2)} = -1$

Activation Function: Step Function

$$f(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

Forward Propagation

$$h = f(W^{(1)}x + b^{(1)}), \quad y = f(W^{(2)}h + b^{(2)})$$

Hidden Layer Computation - What It Solves

Hidden Neuron 1 (h_1): $f(1 \cdot x_1 + 1 \cdot x_2 + 0.5)$

Hidden Neuron 2 (h_2): $f(1 \cdot x_1 + 1 \cdot x_2 + 1.5)$

x_1	x_2	h_1 input	h_2 input	(h_1, h_2)
0	0	0.5	1.5	(1, 1)
0	1	1.5	2.5	(1, 1)
1	0	1.5	2.5	(1, 1)
1	1	2.5	3.5	(1, 1)

Correction: Let's recompute properly

Correct Hidden Layer Transformation

x_1	x_2	h_1	h_2	(h_1, h_2)
0	0	$f(0.5) = 1$	$f(1.5) = 1$	(1, 1)
0	1	$f(1.5) = 1$	$f(2.5) = 1$	(1, 1)
1	0	$f(1.5) = 1$	$f(2.5) = 1$	(1, 1)
1	1	$f(2.5) = 1$	$f(3.5) = 1$	(1, 1)

Problem: All inputs map to (1,1)!

Working Weight Assignment: $W^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$, $b^{(1)} = \begin{bmatrix} -1.5 \\ -0.5 \end{bmatrix}$

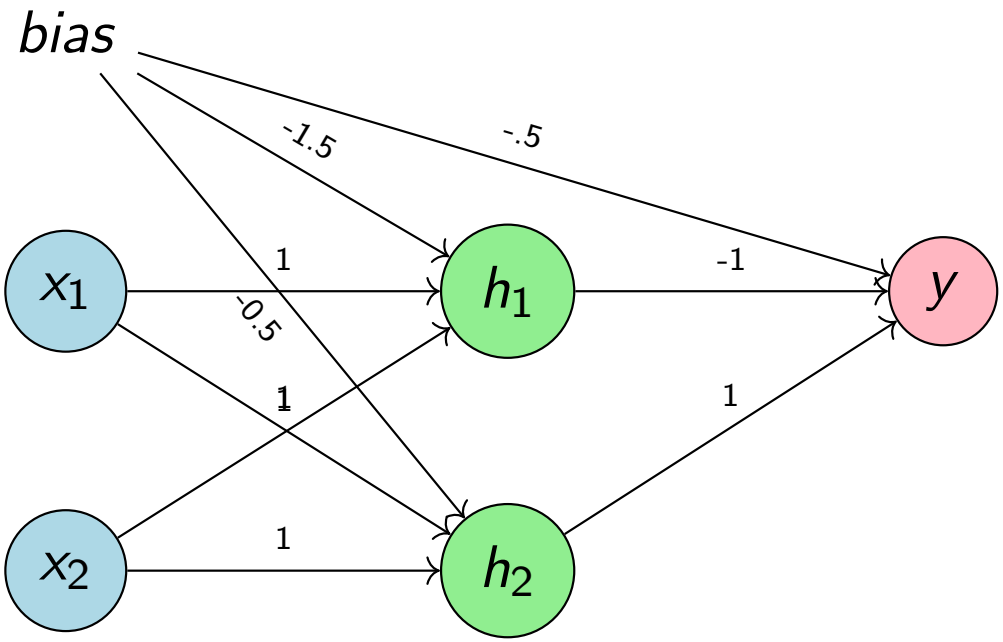
Working Solution with Correct Weights

x_1	x_2	h_1	h_2	(h_1, h_2)
0	0	$f(-1.5) = 0$	$f(-0.5) = 0$	(0, 0)
0	1	$f(-0.5) = 0$	$f(0.5) = 1$	(0, 1)
1	0	$f(-0.5) = 0$	$f(0.5) = 1$	(0, 1)
1	1	$f(0.5) = 1$	$f(1.5) = 1$	(1, 1)

Hidden Neuron	**What It Detects**
h_1	AND function: x_1 AND x_2
h_2	OR function: x_1 OR x_2

(h_1, h_2)	Output computation	y
(0, 0)	$f(-1 + 1 \cdot 0 - 2 \cdot 0) = f(-1) = 0$	0
(0, 1)	$f(-1 + 1 \cdot 0 - 2 \cdot 1) = f(-3) = 0$	Wait, this is wrong!

Complete Working XOR Solution



Final Computation

x_1	x_2	h_1 (AND)	h_2 (OR)	y (XOR)
0	0	0	0	$f(-0.5) = 0$
0	1	0	1	$f(0.5) = 1$
1	0	0	1	$f(0.5) = 1$
1	1	1	1	$f(-0.5) = 0$

What the Hidden Layer Solves

Hidden Neuron 1: AND Function

- Weights: $[1, 1]$, Bias: -1.5
- Detects when **both** inputs are 1
- $h_1 = 1$ only when $x_1 = 1$ AND $x_2 = 1$

Hidden Neuron 2: OR Function

- Weights: $[1, 1]$, Bias: -0.5
- Detects when **at least one** input is 1
- $h_2 = 1$ when $x_1 = 1$ OR $x_2 = 1$

Output Neuron: XOR

- Weights: $[-1, 1]$, Bias: -0.5
- Combines: OR - AND = XOR
- $y =$ (x_1 OR x_2) AND NOT (x_1 AND x

Key Insight

- Hidden layer creates **new feature space**
- Non-linearly separable $\rightarrow$ linearly separable
- Enables solving complex problems

Common activation & loss functions

- Sigmoid: $\sigma(z) = (1 + e^{-z})^{-1}$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- ReLU: $\mathrm{ReLU}(z) = \max(0, z)$, derivative: $1_{z>0}$.
- Softmax for outputs (vector): $\mathrm{softmax}(z)_i = \dfrac{e^{z_i}}{\sum_j e^{z_j}}$.
- Loss examples:
  - MSE: $\mathcal{L}(y, \hat{y}) = \frac{1}{2}|y - \hat{y}|^2$.
  - Cross-entropy (classification, softmax): $\mathcal{L} = -\sum_i y_i \log \hat{y}_i$.

Forward pass (vectorized)

For batch size m:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]} \mathbf{1}_m^\top \quad \in \mathbb{R}^{n_l \times m}$$
$$A^{[l]} = \sigma^{[l]}(Z^{[l]})$$

where $\mathbf{1}_m \in \mathbb{R}^m$ is a vector of ones (used to broadcast bias across the batch).

Goal of Backpropagation

Compute gradients $\frac{\partial \mathcal{L}}{\partial W^{[l]}}$ and $\frac{\partial \mathcal{L}}{\partial b^{[l]}}$ efficiently using the chain rule. Key idea: define the *local (induced) error* $\delta^{[l]} := \frac{\partial \mathcal{L}}{\partial Z^{[l]}} \in \mathbb{R}^{n_l \times m}$.

Output Layer: Derivation of $\delta^{[L]}$ (scalar case)

Consider scalar output $\hat{y} = a^{[L]} = \sigma(z^{[L]})$ and MSE loss $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$.

$$\frac{\partial \mathcal{L}}{\partial a^{[L]}} = -(y - \hat{y})$$

$$\frac{\partial a^{[L]}}{\partial z^{[L]}} = \sigma'(z^{[L]})$$

$$\Rightarrow \quad \delta^{[L]} = \frac{\partial \mathcal{L}}{\partial z^{[L]}} = \frac{\partial \mathcal{L}}{\partial a^{[L]}} \frac{\partial a^{[L]}}{\partial z^{[L]}} = -(y - \hat{y}) \odot \sigma'(z^{[L]})$$

Output Layer: Vector/Softmax + Cross-Entropy (convenience)

For softmax + cross-entropy, the derivative simplifies:

$$\mathcal{L} = -\sum_i y_i \log \hat{y}_i, \quad \hat{y} = \mathrm{softmax}(z) \;\Rightarrow\; \delta^{[L]} = \hat{y} - y.$$

This is a useful and common simplification used in practice.

Hidden Layers: Derivation of $\delta^{[l]}$

Use chain rule. For layer l (not output):

$$\begin{aligned}
\delta^{[l]} &= \frac{\partial \mathcal{L}}{\partial Z^{[l]}} \\
&= \left(W^{[l+1],\top}, \frac{\partial \mathcal{L}}{\partial Z^{[l+1]}} \right) \odot \sigma'(Z^{[l]}) \\
\Rightarrow \quad \delta^{[l]} &= \left(W^{[l+1],\top} \delta^{[l+1]} \right) \odot \sigma'(Z^{[l]})
\end{aligned}$$

where $\odot$ denotes element-wise (Hadamard) product.

Gradients for Weights and Biases (matrix form)

For a batch of size m:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \frac{1}{m} \cdot \delta^{[l]} \cdot A^{[l-1],\top} \quad \in \mathbb{R}^{n_l \times n_{l-1}} \qquad \frac{\partial \mathcal{L}}{\partial b^{[l]}} = \frac{1}{m} \cdot \delta^{[l]} \cdot \mathbf{1}_m \quad \in \mathbb{R}^{n_l}$$

Explanation: each column of $A^{[l-1]}$ is an input example; multiplying $\delta^{[l]}$ by $A^{[l-1],\top}$ sums outer-products across batch.

Weight update (SGD / Mini-batch)

With learning rate η and optionally momentum or Adam, basic SGD update is:

$$W^{[l]} \leftarrow W^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial W^{[l]}}$$

$$b^{[l]} \leftarrow b^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial b^{[l]}}$$

For vectorized mini-batch gradients use the averaged gradients shown earlier.

Backpropagation Algorithm (vectorized)

Given input batch $A^{[0]}$ and labels Y:

1. FORWARD: compute $Z^{[l]}, A^{[l]}$ for $l = 1 \dots L$.
2. OUTPUT error: compute $\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial A^{[L]}} \odot \sigma'(Z^{[L]})$ (or $\hat{Y} - Y$ for softmax+CE).
3. BACKPROP: for $l = L - 1$ down to 1 compute $\delta^{[l]} = (W^{[l+1]\top} \delta^{[l+1]}) \odot \sigma'(Z^{[l]})$.
4. GRADIENTS: $\nabla_{W^{[l]}} = \frac{1}{m} \delta^{[l]} A^{[l-1]\top}, \quad \nabla_{b^{[l]}} = \frac{1}{m} \delta^{[l]} \mathbf{1} * m.$
5. UPDATE: $W^{[l]} \leftarrow W^{[l]} - \eta \nabla_{W^{[l]}}$, $b^{[l]} \leftarrow b^{[l]} - \eta \nabla_{b^{[l]}}$.

Worked example (dimensions)

Suppose: $n_0 = 4$ (input), $n_1 = 3$ (hidden), $n_2 = 2$ (output), batch $m = 5$.

- $W^{[1]} \in \mathbb{R}^{3\times 4}$, $A^{[0]} \in \mathbb{R}^{4\times 5}$, so $Z^{[1]} = W^{[1]}A^{[0]} + b^{[1]}\mathbf{1} * 5^\top$ gives 3×5.
- $\delta^{[2]}$ is 2×5, gradient $\nabla * W^{[2]} = \frac{1}{5}\delta^{[2]}A^{[1]\top}$ yields 2×3.

Numerical stability practical notes

- Use stable softmax with $z' = z - \max(z)$ before exponentiation.
- For cross-entropy with softmax, use the $\hat{y} - y$ shortcut to avoid numeric issues.
- Regularization: add $\frac{\lambda}{2m}|W|_F^2$ to loss; gradient adds $\frac{\lambda}{m}W^{[l]}$.
- Batch normalization changes the gradient flow (requires extra derivatives).

Appendix: chain rule expansion (scalar path)

For clarity, scalar chain rule for weight $w_{ij}^{[l]}$:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{[l]}} = \frac{\partial \mathcal{L}}{\partial z_i^{[l]}} \cdot \frac{\partial z_i^{[l]}}{\partial w_{ij}^{[l]}} = \delta_i^{[l]} \cdot a_j^{[l-1]}.$$

Summing across examples yields the matrix product form used earlier.

Appendix: backprop proof sketch

Start from $\delta^{[L]} = \partial \mathcal{L} / \partial Z^{[L]}$. For earlier layer:

$$\frac{\partial \mathcal{L}}{\partial Z^{[l]}} = \sum_k \frac{\partial \mathcal{L}}{\partial Z_k^{[l+1]}} \frac{\partial Z_k^{[l+1]}}{\partial Z^{[l]}}.$$

Note $Z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$ and $a^{[l]} = \sigma(Z^{[l]})$, which yields the formula $\delta^{[l]} = (W^{[l+1]\top} \delta^{[l+1]}) \odot \sigma'(Z^{[l]})$.

References and further reading

- Michael Nielsen, "Neural Networks and Deep Learning" (chapter on backprop).
- Goodfellow, Bengio, Courville, "Deep Learning" (2016) – Backprop chapter.
- Various online lecture notes (Stanford CS231n): vectorized backprop derivations.

Questions

Thank you! Any questions?